





EXCEL

Introduction to Python in Excel

Training Objectives



1. Understand how Python integrates with Excel.
2. Identify use cases where Python adds value beyond traditional Excel formulas.
3. Use Python directly within Excel cells. Manipulate and analyze Excel data using Python libraries.
4. Create advanced visualizations and summaries.
5. Automate repetitive analytical tasks.
6. Determine when to use Excel formulas, Power Query, VBA, or Python.

WHAT IS PYTHON ?



- Python is a high-level, general-purpose programming language created by Guido van Rossum and first released in 1991. It was designed with a strong emphasis on readability, simplicity, and productivity, making it accessible to beginners while remaining powerful enough for professional software development.

Key Characteristics of Python	Python is used in many domains
Easy to learn and read	Data analysis and business intelligence
Cross-platform (Windows, Mac, Linux)	Artificial Intelligence (AI) and Machine Learning
Open-source and free	Automation and scripting
Large community support	Web development
Thousands of specialized libraries	Scientific computing
Strong integration with business and analytical tools	Financial modeling
	Data engineering

WHY IS PYTHON SO POPULAR ?



- Python combines simplicity, flexibility and a powerful ecosystem of libraries.
 - Pandas: Data manipulation and analysis
 - NumPy : Numerical computing
 - Matplotlib : Data visualization
 - Seaborn : Statistical graphics
- Handles large and complex datasets
Data cleaning, transformation, validation, Statistical and predictive analysis.
- Automation capabilities
Cleaning files, combining reports, Updating calculation, Producing monthly summaries
- Advanced Analytics and Statistics
Regression analysis, Forecasting, Clustering, Anomaly detection, Machine learning, Time-series analysis



- Microsoft's message:
 - Excel formulas and Power Query remain essential.
 - VBA remains useful.
 - Python adds a new advanced analytics layer.
- Microsoft views Python as a strategic technology because it is the dominant language used in AI and Data-Driven Decision Making.
Python into Excel makes advanced analytics accessible to a much broader audience.

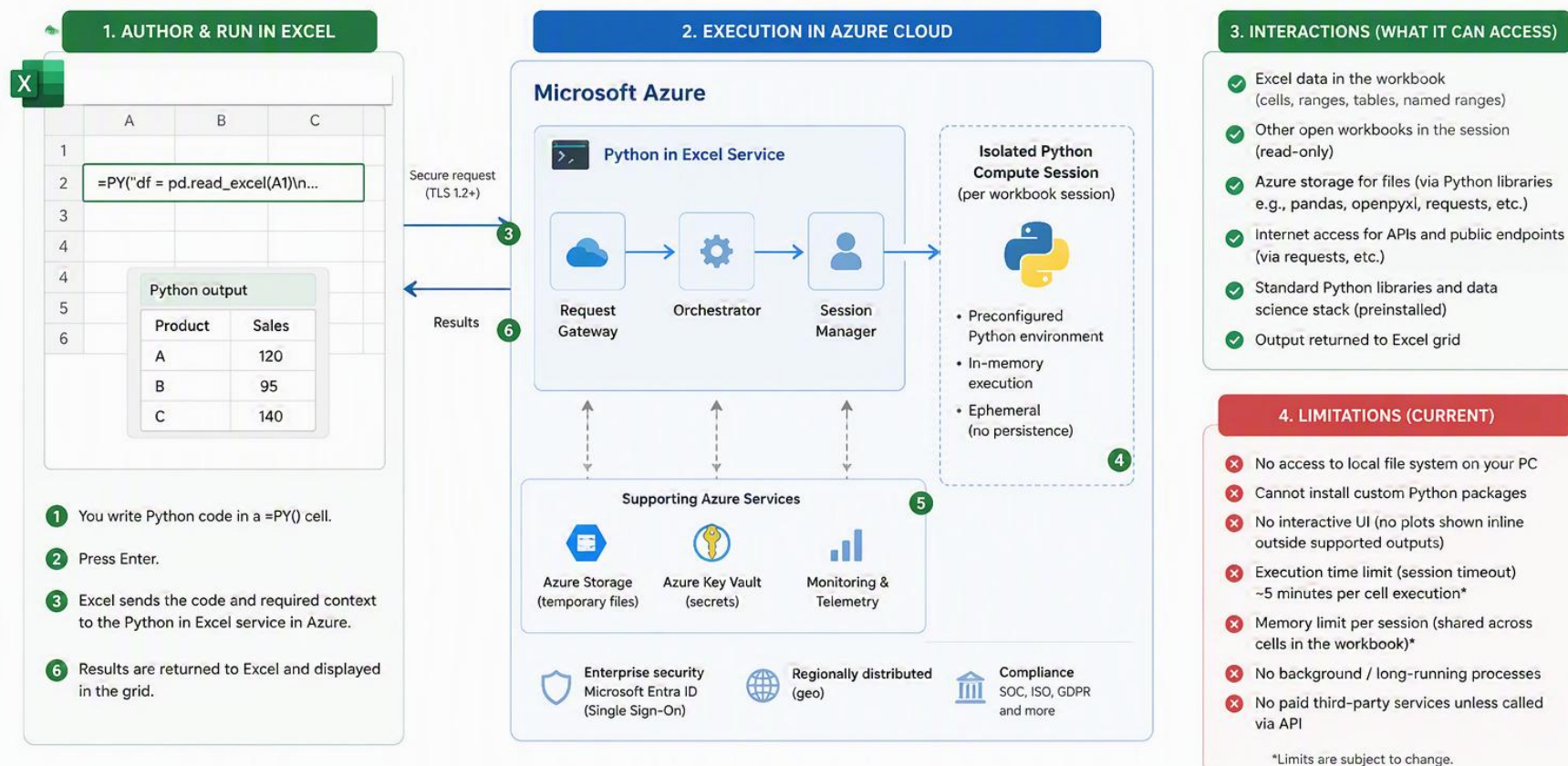
Python in Excel transforms Excel from a spreadsheet tool into a more powerful analytics platform, allowing business users to perform advanced data analysis, visualization, and automation without leaving their familiar Excel environment.

How Python works within Excel



PYTHON IN EXCEL – HOW IT WORKS

Execution flow, interactions and Azure cloud architecture



EXECUTION FLOW SUMMARY

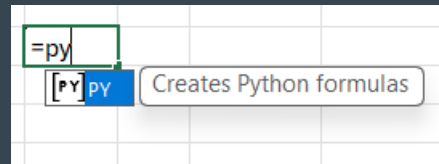


Your data and code are encrypted in transit and at rest in Microsoft Azure. Microsoft does not use your data or code to train models.

Getting started with Python in Excel



- The Excel function `PY()` will allow you to write Python code in an Excel cell.
- Simply type `=PY()` and the cell will display a Python icon meaning that you can enter Python code into the formula bar.
- Alternatively, use the keyboard shortcut `ALT+CTRL+SHIFT+P`



- You can also use the Python editor available here: [Formulas > \(Python\) Editor](#)

Use **CTRL + ENTER** to validate the changes in your Python code.

Exercise 1



- a. Enter any number in cell A6 then in cell C6, type the following Python code:

```
a , b = 10,20  
c = xl("A6")  
mysum=a+b+c  
myavg = mysum / 3  
myavg
```

- b. In cell C9, type the following Python code :

*Notice the use of a cell, a cell in a specified sheet and a defined name
Use # for comments in Python*

```
# cent est un nom défini dans le classeur  
  
xl("A6")+xl("Variables!E2")+xl("cent")
```

Exercise 2



- a. In cell C12, type the following Python code:

```
a = xl("A6")
def square(x):
    return x*x
txt1 = " square of "
f"{txt1}{a} = {square(a)}"
```

Use 'def' to define an internal function (similar to VBA). The function returns the value following the 'return' keyword. Indent all code inside the function.

Use the f function for complex output

- b. The f before a string creates an f-string (formatted string literal). It lets you embed variables or expressions directly inside a string using curly braces {}.
- Test the following code in E12 (wrap text in cell)

```
X, y = 10,5
price = 12.345678
name = "python"
```

```
f"{x} + {y} = {x + y}\nThis is {name.upper()}\nPrice: {price:.2f} "
```

Exercise 3



- Using the Pandas library:

```
liste = [10,20,xl("A6")]  
pd.Series(liste).mean()
```

Type this code in cell E6

- Verify the Python initializations: Formulas > (Python) Initialization

```
1  import numpy as np  
2  import pandas as pd  
3  import matplotlib.pyplot as plt  
4  import seaborn as sns  
5  import statsmodels as sm  
6  import excel  
7  import warnings
```

Exercise 4: Working with a list



- Create a list object in Python containing the value in the cell A2, the content of the defined name 'VATrate' and the values in A4 and B4 from the sheet named 'Variables'.
- Create the necessary Python code that displays in cell A16 the following string:

Ex4

Tax rate : 8.10% *** VIP discount : 10.25% // Rounded discount: 10%

SOLUTION

```
liste_texte = [xl("Variables!A2"),xl("VATrate"),xl("Variables!A4"),xl("Variables!B4")]
tvaformatted = format(liste_texte[1],".2%")
viprounded = format(round(liste_texte[3],2),".0%")
f"{liste_texte[0]} : {tvaformatted} *** {liste_texte[2]} : {liste_texte[3]: .2%} // Rounded discount:
{viprounded}"
```

Conditional branching (IF)



- Using the IF, ELIF, ELSE function

```
score = 75

if score >= 90:
    print("Grade A")
elif score >= 80:
    print("Grade B")
elif score >= 70:
    print("Grade C")
else:
    print("Grade D")
```

- Using the AND, OR function inside IF

```
is_weekend = False
is_holiday = True
Result = ""

if is_weekend or is_holiday:
    result = "No work today"
elif not is_weekend and not is_holiday:
    result = "today is a week day"
```

Don't forget the indentation to define blocks of code

In Python, empty strings, lists, dictionaries, and None evaluate to False.

Exercise 5



- Create a python function named 'testmyval' that evaluates if the amount in cell A6 is positive, negative or equal to 0.

```
x = xl("A6")
def testmyval(myval):
    if myval > 0: return "Positive"
    elif myval < 0: return "Negative"
    else: return "Zero"
testmyval(x)
```

SOLUTION

```
x = xl("A6")
"Positive" if x > 0 else "Negative" if x < 0 else "Zero">
```

Exercise 6e



- Ex6e: Create a Python code that fetches the values in E2:N2 from the sheet 'Variables' then loop through the values and multiply each value by 10 if the original number is greater than 5, else simply return the value itself in the results[] list. Eventually, use the join() method to display all the resulting values separated by a comma in a string called final_string.

SOLUTION

```
vals = xl("Variables!E2:N2")
results=[]
final_string =""
```

```
for v in vals:
```

```
    # If the value v in the array vals is greater than 5 then store v *10, else simply v
```

```
    if v > 5:
```

```
        results.append(v* 10)
```

```
    else:
```

```
        results.append(v)
```

```
final_string =", ".join(str(x) for x in results if x%2 == 0)
```

```
# the join() method places a ',' between each element of results[] if they are even numbers
```

```
f"output: {final_string}"
```

ExD1: Working with dates



- Use the `to_datetime` function from the pandas library to convert most of the date formats to a standard Excel date format.
- Look in the sheet 'Dates in Python' and convert the pseudo-dates in A5:A16
- Example of use:
`pd.to_datetime(xl("A5"))`

https://pandas.pydata.org/docs/reference/api/pandas.to_datetime.html

ExD2: Calculating remaining days



In order to use the Date and time functions in Python, import the date, datetime and timedelta modules from the datetime library.

Add the following code to the initialization code of your workbook:

```
from datetime import date, datetime, timedelta
```

- Once this is done, calculate the remaining number of days till your next birthday.

```
today = date.today()
birthday = date(2026, 8, 10)

days_remaining = (birthday - today).days

f"Days remaining: {days_remaining}"
```

ExD3: Working days



- Calculate the remaining number of working days till the end of this year.
- Analyze the following code and test it inside Excel:

```
today = date.today()
end_of_year = date(today.year, 12, 31)
working_days = 0
current = today

while current <= end_of_year:

    if current.weekday() < 5:
        working_days += 1

    current += timedelta(days=1)

f"There are {working_days} working days remaining this year."
```

Working with Dataframes



Pandas is the most widely used Python library for working with tabular data, and it is the foundation of most data analysis performed in Python in Excel.

When using 2D ranges in excel, Python returns a pandas DataFrame by default.

Example: `df = xl("A1:C100", headers=True)` # df is a pandas.DataFrame

Pandas is like Excel tables, formulas, filters, PivotTables, Power Query, and VLOOKUP/XLOOKUP combined into a programmable toolkit.

<https://pandas.pydata.org/docs/reference/index.html>

Working with Dataframes



- Create a dataframe from an Excel table located in the 'Variables' sheet:

```
df_service=xl("tbl_service[#All]", headers=True)
f"Creating the df_service dataframe"
```

- A dataframe should be created before any calculation is applied onto its Series.
- Good practice: define your dataframe in the top left cells of the active sheet.
- Ex DF1: Use the describe function to display detailed statistics about your dataframe.

```
df_service.describe()
```

- Ex DF2: use the shape() and the columns() functions to display the size and the list of columns of the dataframe.

```
df_service.shape
```

Working with Dataframes



- Ex DF3: filter the dataframe (use filter ()) to view rows where 'date_opened' is between january 15th and 17th 2019.

```
df_service[(df_service['date_opened']>='2019-01-15') & (df_service['date_opened']<'2019-01-17')]
```

- Ex DF4: Calculate the average value, the total sum of [m_cached_price] and the number of sales for the df_service.

```
df_service["m_cached_price"].mean()  
df_service["m_cached_price"].sum()  
df_service["m_cached_price"].count()
```

- Ex DF5: use the groupby() to summarize the average of [m_cached_payed] by [m_nb_customer]. This creates a pivot table with Python.

```
df_service.groupby('m_nb_customer')['m_cached_payed'].mean()
```

Working with Dataframes



- Ex DF6: Add 3 columns to df_service, one for the date (in days), one for the month and one for the year. Then summarize df_service showing the sum of [m_cached_price] by Year and Month. Display the new df_service.

```
# Add 3 columns to df_service to create the date, year and month columns from the date/time column 'date_opened'
df_service["Date"] = pd.to_datetime(df_service["date_opened"])
df_service["Year"] = df_service["Date"].dt.year
df_service["Month"] = df_service["Date"].dt.month

# pivoting by Year and Month, summing the 'm_cached_price' column
MonthlyTurnover = df_service.groupby(["Year", "Month"])["m_cached_price"].sum()

# Display the pivot table
MonthlyTurnover
```

- Display the list of columns for the df_service

```
df_service.columns
```

Working with Dataframes



- Ex DF7: Select the following column ["date_opened","id_store","m_cached_payed"] and use `sort_values()` to sort descending on the 3rd column. Keep the top 10 rows of the sorted dataframe and do not display the standard index, use `reset_index()` to do so.

```
# Step 1 : Display 3 columns from the original table
#df_service[["date_opened","id_store","m_cached_payed"]]

# Step 2 : Sort descending on 'm_cached_payed'
#df_service[["date_opened","id_store","m_cached_payed"]].sort_values(by="m_cached_payed",
ascending=False)

# Step 3 : Keep top 10 values (head(10)) of the sorted table and do not display the index
(reset_index)
df_service[["date_opened","id_store","m_cached_payed"]].sort_values(by="m_cached_payed",
ascending=False).head(10).reset_index(drop=True)
```

Working with Dataframes



- Ex DF8: Display the unique values of ["id_store"] in df_service.

```
df_service["id_store"].unique()
```

- Ex DF10: Summarize df_service by Year and Month and add 4 aggregations: the sum of [m_cached_payed] and [m_cached_price], the average of [m_nb_customer] and the count of [id_order].
Round all the results to 0 decimals.
Sort the resulting pivot table by Year and month

Working with Dataframes



- Ex DF10: Solution

```
pivot= (
df_service
# create the pivot table with these 2 dataframe cols in the rows section
    .groupby(["Year","Month"])
# Add 4 aggregations columns
    .agg(
        {"m_cached_payed": "sum",
         "m_cached_price": "sum",
         "m_nb_customer": "mean",
         "id_order": "count"})
).round(0)    #Rounds the calculated values to the unit

# Rename the columns of the pivot table
pivot = pivot.rename(columns={
                        "m_cached_payed": "Total Paid",
                        "m_cached_price": "Total Price",
                        "m_nb_customer": "AVG Clients",
                        "id_order": "Nb Sales"
                        })
# Display the pivot table and sort ascending on 'Year' and 'Month'
pivot.sort_values(by=["Year","Month"])
```

Merging Dataframes



```
DataFrame.merge(right, how='inner', on=None, left_on=None, right_on=None, left_index=False, right_index=False, sort=False, suffixes=('_x', '_y'), copy=<no_default>, indicator=False, validate=None)
```

right

Object to merge with.

how{'left', 'right', 'outer', 'inner', 'cross', 'left_anti', 'right_anti'},

On

Column or index level names to join on.

Suffixes

A length-2 sequence where each element is optionally a string indicating the suffix to add to overlapping column names in *left* and *right* respectively.

- `merge()` matches rows between two tables using a common key column and brings the related information together into a new DataFrame.
- It's similar to VLOOKUP or XLOOKUP or the Power Query merge function.

Merging Dataframes



In the 'cleaning and fusion' sheet:

- Ex MD1:

Create a dataframe based on cells in A4:D18 in cell E5.

Create a new column [AVG] calculating the average sale per customer row by row.

```
dfsmall = xl("A4:D18", headers=True)
# Create a column [AVG] calculating the sales average on every row
dfsmall['AVG']=dfsmall['Amount']/dfsmall['NB_cust']
f'Creating the dfsmall dataframe'
```

- Ex MD2:

Use `dropna()` to get rid of empty rows in a new dataframe `dfclean`.

```
#dfclean = dfsmall.dropna(how='all').reset_index(drop=True)
dfclean
```

- Ex MD3:

Define 2 dataframes on ranges A21:B29 and A31:C39 then merge the 2 over the column [Order ID]

Merging Dataframes



In the 'cleaning and fusion' sheet:

- Ex MD3:

Define 2 dataframes on ranges A21:B29 and A31:C39 then merge the 2 over the column [Order ID]

```
#Define 2 dataframes
left=x1("A21:B29", headers=True)
right=x1("A31:C39", headers=True)

#Merge those 2 dataframes over [Order ID]
#suffixes will add a suffix to identical columns (if any)
fusion=left.merge(right,how='left',on='Order ID', suffixes=('_1','_2'))
fusion
```

Detecting anomalies in a Dataframe



Ex MD4:

1. Create a dataframe `df_anomaly` from cells A4:D12.
2. Add a new column `[avg/cust]` calculating the average sale by customer.
3. Calculate the global average and the standard deviation over this new column in 2 variables `avg` and `stddev`.
4. Add a new column `[zscore]` that divides `[avg/cust]` by `stddev`.
5. Add another column `[anomaly]` that displays `True` if `[zscore]` is greater than 2.

Unpivoting with Python (using Pandas)



```
DataFrame.melt(id_vars=None, value_vars=None, var_name=None, value_name='value', col_level=None, ignore_index=True)
```

id_vars

Column(s) to use as identifier variables.

value_vars

Column(s) to unpivot. If not specified, uses all columns that are not set as *id_vars*.

var_name

Name to use for the 'variable' column. If None it uses `frame.columns.name` or 'variable'.

value_name

Name to use for the 'value' column, can't be an existing column label.

In sheet 'Unpivot in Python'

Ex UPI: Use the `melt()` function to unpivot the Q1..Q4 columns in Table1.

[Training] and [Level] are the identifier columns. The new unpivoted columns should be called [Q] and [NB].

Unpivoting with Python



In sheet 'Unpivot in Python'

Ex UPI: Solution

```
pd.melt(xl("Table1[#A11]", headers=True), id_vars=['Training', 'Level'], var_name='Q', value_name='NB')
```

Excel vs Python charts



Quick, ad hoc analysis
Small datasets
Dashboards for business users
Interactive filtering with Pivot Tables and Slicers
Users who don't know Python



Chart creation automation
Large Data volumes
Advanced visuals (violin, heatmaps, etc)
Data preparation and visualization together
Consistency
Integration into Data pipelines



Use Excel for exploration. Use Python for automation.

Plotting with Matplotlib in Python



- **Matplotlib is the standard Python plotting library**
- **Used for creating static charts and data visualizations**
- **Works very well with Pandas DataFrames**
- **Foundation for libraries like Seaborn**
- **Imported by default in Excel Python as plt**

Basic workflow:

```
df.plot(x="Month", y="Sales")  
plt.show()
```

The figures and Axes concept:

```
fig, ax = plt.subplots()
```

```
ax.plot(df["Month"], df["Sales"])  
ax.set_title("Monthly Sales")  
plt.show()
```

- The figure is similar to the whole chart window
- The ax is similar to a chart plot area
- You can define several axes in the same chart

Plotting with Matplotlib in Python



Most common chart types:

Line chart:	<code>df.plot(df["Month"], df["Sales"])</code>
Bar chart:	<code>plt.bar(df["Product"], df["Revenue"])</code>
Horizontal bar:	<code>plt.barh(df["Product"], df["Revenue"])</code>
Scatter plot:	<code>plt.scatter(df["Age"], df["Salary"])</code>
Histogram:	<code>plt.hist(df["Salary"], bins=20)</code>
Pie chart:	<code>plt.pie(df["Value"], labels=df["Category"])</code>

Essential formatting:

Title:	<code>plt.title("Monthly Sales")</code>
Axis labels:	<code>plt.xlabel("Month")</code> <code>plt.ylabel("Sales")</code>
Grid:	<code>plt.grid(True)</code>
Legend:	<code>plt.legend()</code>
Figure size:	<code>plt.figure(figsize=(8, 4))</code>
Colors/Style:	<code>plt.plot(df["Sales"],</code> <code>color="blue",</code> <code>linestyle="--",</code> <code>marker="o")</code>

Plotting with Matplotlib in Python



Subplots (display several charts together):

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
```

```
axes[0].plot(df["Sales"])
```

```
axes[1].hist(df["Sales"])
```

```
plt.show()
```

Essential formatting:

Title: `plt.title("Monthly Sales")`

Axis labels: `plt.xlabel("Month")`

`plt.ylabel("Sales")`

Grid: `plt.grid(True)`

Legend: `plt.legend()`

Figure size: `plt.figure(figsize=(8, 4))`

Colors/Style: `plt.plot(df["Sales"],
color="blue", linestyle="--", marker="o")`

Plotting with Matplotlib in Python



Exercise MPL1:

1. In A1: Create a dataframe from the data in A4:D14 (sheet 'Matplotlib')
2. In F4: Find the max value for [amount] in the dataframe (Amount_max) and the associated Store ID (StoreIDMax).
3. Create ListOfColors, a list of colors for all columns: red when the Amount matches the Amount_max value, steelblue for others.
4. Create a bar chart representing [Amount] per [id_store] using the list of colors.
 - a) Add title 'Amount per store' and customize the legend as 'Amount [MCHF]'.
 - b) Add title (label) on X axis 'Store reference number'.
5. Add a second chart, a line chart representing [position] per [id_store].
 - a) Change color to dark green and markers to 'o'.
 - b) Replace the values on the X axis by the values of [id_store]

```
# Find index of maximum amount
Amount_max = df_resto["Amount"].idxmax()
# Find the matching store ID
StoreIDMax = df_resto.loc[Amount_max, "id_store"]
# ListOfColors : red for StoreIDMax, steelblue for the
others
ListOfColors = [ "red" if __id == StoreIDMax else
"steelblue"   for __id in df_resto["id_store"]
]
# generate the bar plot Amount per Store ID
df_resto.plot(
    x='id_store',
    y='Amount',
    kind='bar',
    title='Amount per Store',
    color=ListOfColors,
    label="Amount [MCHF]",
    xlabel="Store reference number"
)
# generate the line chart for the NB orders
linechart = df_resto.plot(
    x='position',
    y='NB orders',
    kind='line',
    title='Number of orders',
    marker="o",
    color='darkgreen',
    xlabel="Store ID"
)
# Change the horizontal axis labels
# by default we set it to [position]
linechart.set_xticklabels(df_resto["id_store"])
```

Plotting with Matplotlib in Python



Exercise MPL2:

1. Recreate both charts from Exercise MPL1 but display both charts on the same figure.
2. Set the size of the figure to (10,5)
3. Create a duplicate Y axis
4. Disable the grid for both charts.

```
fig, ax1 = plt.subplots(figsize=(10, 5))

# Amount (bars)
df_resto.plot(
    x="id_store",
    y="Amount",
    kind="bar",
    color=ListOfColors,
    ax=ax1
)

# NB orders (line on secondary axis)
ax2 = ax1.twinx()

ax2.plot(
    df_resto.index,
    df_resto["NB orders"],
    color="darkgreen",
    marker="o",
    linewidth=2,
    label="NB orders"
)

ax1.set_title("Amount and NB Orders per Store")
ax1.set_ylabel("Amount")
ax1.grid(False)
ax1.set_xticklabels(df_resto["id_store"], rotation=45)

ax2.set_ylabel("NB Orders")
ax2.grid(False)

#plt.tight_layout() #optimize layout
plt.show()
```

Recap exercise



With the data in sheet 'RECAP':

1. Create a dataframe `df_recap` that stores the data in table3.
2. Create a new column that calculates `[Revenue]` ($=\text{Qty} * \text{price}$) row by row.
3. Summarize the data by summing `[Revenue]` by regions.
4. Create a `kpis` dataframe displaying 4 columns: the sum of `[Revenue]`, the global average, the max and min values of `[Revenue]`.
5. Summarize the sum of revenue by products and sort descending.
6. Create a bar plot showing the Revenue by region.
7. Filter `df_recap` to identify the outliers (where Revenue is in the top 5%).
8. Create a dataframe that displays the Best selling region, the best selling product and the global Revenue.

Bonus exercise



Analysis:

1. Which region contributes more than 25% of total revenue?AN2:
2. What would revenue become with a 5% price increase?
3. Create a dataframe displaying the ranking of products revenues.

Business scenario:

1. Management wants every transaction classified according to its revenue. 'Excellent' if [Revenue] ≥ 5000 , 'Good' if ≥ 1000 , standard otherwise
2. Identify all regions with sales below a target of 5000.
3. Create a bonus salary dataframe that just displays the total amount of bonuses paid. (10% if Revenue ≥ 5000 , 5% if ≥ 2000 and 2% otherwise).



Your feedback:

<https://learn.digicomp.ch>

<https://digicomp.my.salesforce-sites.com/EKBF?lang=fr>

